```
"""
screening.py
============
The cheap, deterministic filter that runs BEFORE any Claude call.

Purpose: never spend an API token, and never generate an LOI, for a property
that fails a hard rule. Rules here are boolean and auditable. Judgment calls
(is the master bedroom really shareable? is that slope a problem?) are left to
afh_evaluator.py.

County determination
--------------------
ZIP codes do not align cleanly with county boundaries. This module treats a
ZIP hit as *evidence*, not proof:
  - ZIP in CLACKAMAS_ZIPS      -> pass, county = "Clackamas"
  - ZIP in STRADDLE_ZIPS       -> pass WITH a warning to verify the parcel
  - city in CLACKAMAS_CITIES   -> pass with lower confidence
  - neither                    -> fail
Before you sign anything, verify the parcel against the Clackamas County
Assessor / A&T property records. Zoning and licensing both turn on it.
"""

from __future__ import annotations

import logging
from typing import Optional

import config
from models import Listing, ScreenOutcome, ScreenResult

log = logging.getLogger(__name__)


# ---------------------------------------------------------------------------
# County
# ---------------------------------------------------------------------------

def determine_county(listing: Listing) -> tuple[Optional[str], list[str]]:
    """Return (county_or_None, warnings)."""
    warnings: list[str] = []
    zipc = (listing.zip_code or "").strip()
    city = (listing.city or "").strip().lower()

    if zipc and zipc in config.CLACKAMAS_ZIPS:
```

```
        return "Clackamas", warnings

    if zipc and zipc in config.STRADDLE_ZIPS:
        warnings.append(
            f"ZIP {zipc} straddles a county line. VERIFY the parcel is in "
            "Clackamas County via the county assessor before proceeding."
        )
        return "Clackamas (unverified)", warnings

    if city in config.CLACKAMAS_CITIES:
        warnings.append(
            f"County inferred from city name '{listing.city}' only "
            "(no recognised ZIP). Verify the parcel."
        )
        return "Clackamas (unverified)", warnings

    if zipc and zipc.startswith("97"):
        warnings.append(f"Oregon ZIP {zipc} is not on the Clackamas list.")

    return None, warnings


# ---------------------------------------------------------------------------
# Bedroom logic
# ---------------------------------------------------------------------------

def has_convertible_bonus_room(
    listing: Listing,
    thresholds: Optional[config.ScreeningThresholds] = None,
) -> bool:
    """True if the description mentions a den/bonus/office that could become
    a bedroom. This is a keyword heuristic -- egress windows, ceiling height
    and closet requirements still have to be verified on site."""
    t = thresholds or config.THRESHOLDS
    text = f"{listing.description} {listing.raw_text}".lower()
    return any(kw in text for kw in t.bonus_room_keywords)


def bedroom_check(
    listing: Listing,
    thresholds: Optional[config.ScreeningThresholds] = None,
) -> tuple[bool, list[str]]:
    t = thresholds or config.THRESHOLDS
    notes: list[str] = []

    if listing.bedrooms is None:
        return False, ["Bedroom count unknown -- cannot screen."]
```

```
    if listing.bedrooms >= t.min_bedrooms:
        notes.append(f"{listing.bedrooms:g} bedrooms meets the {t.min_bedrooms:g}
        return True, notes

    if listing.bedrooms >= t.min_bedrooms_with_bonus and has_convertible_bonus_ro
        notes.append(
            f"{listing.bedrooms:g} bedrooms plus a den/bonus room mentioned in th
            "listing -- conditional pass, conversion feasibility unverified "
            "(egress, closet, ceiling height all need on-site check)."
        )
        return True, notes

    return False, [
        f"{listing.bedrooms:g} bedrooms is below the {t.min_bedrooms:g}-bedroom "
        "requirement and no convertible bonus room was mentioned."
    ]


# ---------------------------------------------------------------------------
# Seller motivation
# ---------------------------------------------------------------------------

def motivation_check(
    listing: Listing,
    thresholds: Optional[config.ScreeningThresholds] = None,
) -> tuple[bool, list[str]]:
    t = thresholds or config.THRESHOLDS
    signals: list[str] = []

    if listing.days_on_market is not None and listing.days_on_market >= t.min_day
        signals.append(f"{listing.days_on_market} days on market (>= {t.min_days_

    if listing.price_reduction_count > 0:
        signals.append(f"{listing.price_reduction_count} recorded price reduction

    if listing.price_drop_amount:
        signals.append(f"Price cut of ${listing.price_drop_amount:,} from origina

    if listing.motivation_keywords_found:
        joined = ", ".join(listing.motivation_keywords_found[:5])
        signals.append(f"Motivation language in the listing: {joined}.")

    return bool(signals), signals


# ---------------------------------------------------------------------------
```

```
# Price band
# ---------------------------------------------------------------------------

def price_check(
    listing: Listing,
    thresholds: Optional[config.ScreeningThresholds] = None,
) -> tuple[bool, list[str]]:
    t = thresholds or config.THRESHOLDS
    if listing.list_price is None:
        return True, ["List price unknown -- price band not applied."]
    if listing.list_price > t.max_list_price:
        return False, [f"${listing.list_price:,} exceeds the ${t.max_list_price:,
    if listing.list_price < t.min_list_price:
        return False, [f"${listing.list_price:,} is below the ${t.min_list_price:
    return True, [f"${listing.list_price:,} is inside the target band."]


# ---------------------------------------------------------------------------
# Orchestration
# ---------------------------------------------------------------------------

def screen(
    listing: Listing,
    thresholds: Optional[config.ScreeningThresholds] = None,
) -> ScreenResult:
    """Run every hard rule. Mutates listing.county as a side effect."""
    t = thresholds or config.THRESHOLDS
    reasons: list[str] = []
    warnings: list[str] = []

    county, county_warnings = determine_county(listing)
    warnings.extend(county_warnings)
    listing.county = county
    if county is None:
        return ScreenResult(
            outcome=ScreenOutcome.FAIL_COUNTY,
            reasons=[f"Not identified as Clackamas County ({listing.city} {listin
            warnings=warnings,
        )
    reasons.append(f"County: {county}.")

    ok, notes = bedroom_check(listing, t)
    reasons.extend(notes)
    if not ok:
        outcome = (ScreenOutcome.FAIL_INCOMPLETE if listing.bedrooms is None
                   else ScreenOutcome.FAIL_BEDROOMS)
        return ScreenResult(outcome=outcome, reasons=reasons, warnings=warnings)
```

```
    ok, notes = price_check(listing, t)
    reasons.extend(notes)
    if not ok:
        return ScreenResult(ScreenOutcome.FAIL_PRICE, reasons, warnings)

    ok, notes = motivation_check(listing, t)
    if ok:
        reasons.extend(notes)
    else:
        return ScreenResult(
            ScreenOutcome.FAIL_MOTIVATION,
            reasons + ["No seller-motivation signal (DOM, price cut, or keyword).
            warnings,
        )

    if listing.square_feet and listing.square_feet < t.min_square_feet:
        warnings.append(
            f"{listing.square_feet:,} sq ft is under the "
            f"{t.min_square_feet:,} sq ft preference -- "
            "check usable bedroom area carefully."
        )

    if not listing.agent.is_contactable:
        warnings.append(
            "No listing-agent email found in the alert. The LOI will still be "
            "generated but no Gmail draft can be created until you supply one."
        )

    return ScreenResult(ScreenOutcome.PASS, reasons, warnings)


def screen_all(
    listings: list[Listing],
    thresholds: Optional[config.ScreeningThresholds] = None,
) -> list[tuple[Listing, ScreenResult]]:
    out: list[tuple[Listing, ScreenResult]] = []
    for lst in listings:
        result = screen(lst, thresholds)
        log.info("Screen %-45s -> %s", lst.address[:45], result.outcome.value)
        out.append((lst, result))
    return out
```